Improving Human-Robot Teamwork in Urban Search and Rescue Through Episodic Memory of Prior Collaboration

Taewoon Kim[1], Emma van Zoelen[2], and Mark Neerincx[2]

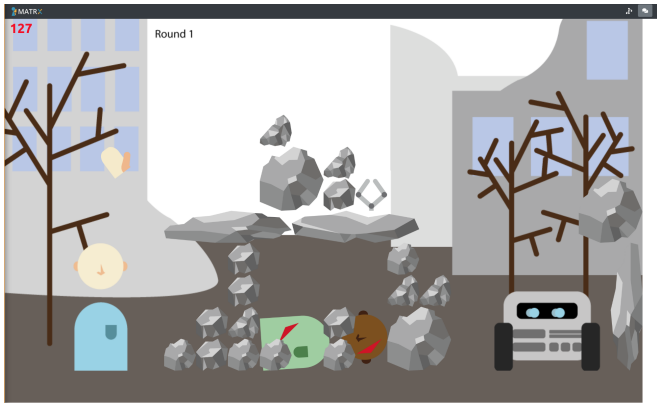


Fig. 1: A screenshot of the MATRX USAR simulation. The human user is shown on the left, while the robot is on the right. In the upper right corner, users can click icons to define a collaboration pattern (CP) or open the chat interface.

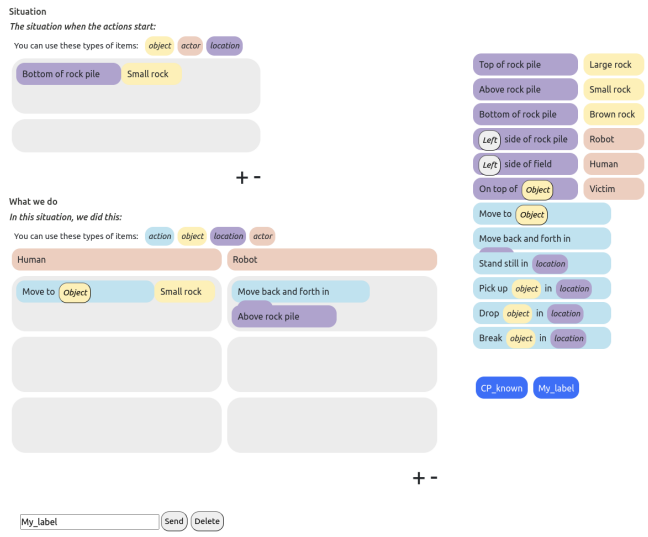
***Abstract*— Effective human-robot teamwork requires robots to adapt to partners, situations, and task dynamics from the start of an interaction. In the MATRX Urban Search and Rescue (USAR) environment, people can externalize collaboration patterns (CPs) they discover during teamwork through a chat and reflection interface. We study whether a robot can use such prior team experience to become a better teammate in future interactions. To this end, we represent historical CPs as knowledge-graph episodic memories and use graph representation learning with a node-classification objective to identify a representative and effective memory for reuse. We then initialize the robot with this memory before a new collaboration episode begins. Across 20 participants and 160 round-level observations, initializing the robot with a single automatically selected prior CP increases rescue success from 25.7% to 41.3% and reduces average task time by 283 seconds. The strongest gains appear at the beginning of interaction, suggesting that reusable episodic memory can help robots enter collaboration with more effective task knowledge and support smoother early teamwork.**

I. Introduction

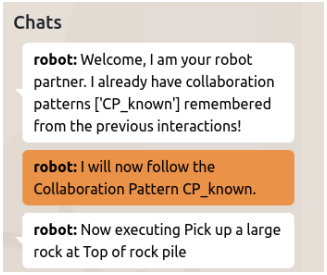
Industrial robots usually work in static settings with limited human contact [1], but labour shortages and hazardous tasks are driving a shift toward teammate-style autonomy [2]. In these settings, robots are expected not only to execute actions correctly, but also to coordinate with humans as partners who have evolving goals, preferences, and working styles [3], [4], [5]. Disaster response is a particularly demanding example: assistive robots are being developed to help rescue teams navigate debris, locate survivors, and deliver supplies in hazardous environments [6], [7].

Despite their potential, assistive robots face substantial challenges in adapting to human partners. Unlike traditional robots operating in controlled environments, they must interpret human intentions, adjust to changing situations, and learn from past interactions. Prior work in human-robot teaming has emphasized the importance of mutual adaptation, shared understanding, and team-level coordination mechanisms that help robots align with their partners over time [3], [8], [5].

One approach to studying these issues is the MATRX



(a) Users can drag and drop boxes (representing objects, actors, and locations) to document the emergent CPs they observe with the robot.



This chat interface enables the robot to communicate updates and inform human users about ongoing events.

(b) The chat interface.

Fig. 2: User interaction features in the MATRX USAR environment. (a) The CP interface. (b) The chat interface.

[1]Taewoon Kim, HumemAI, The Netherlands taewoon@humem.ai
[2]Emma van Zoelen and Mark Neerincx, TNO, The Netherlands emma.vanzoelen@tno.nl, mark.neerincx@tno.nl

Urban Search and Rescue (USAR) environment [9], [10], a virtual simulation where a human and a collaborative robot work together to rescue victims. Previous studies have shown that robots in this setting can adapt to human behavior over time, improving teamwork efficiency. Notably, when human participants can specify collaboration patterns (CPs) and communicate through a chat interface, coordination improves and rescue operations become more effective [11], [10], [12]. In this context, CPs are defined as human-robot actions conditioned on situations (see Fig. 2a).

Shared memory has long been recognized as a key element in fostering mutual understanding between humans and robots [13]. Building on this idea, we treat previously observed CPs as episodic long-term memories of prior collaboration and ask whether a robot can carry forward a useful team experience into a new interaction. To do so, we represent past CPs as knowledge-graph memories and use graph representation learning with node-type supervision to identify a representative and effective memory for reuse. Instead of starting each episode with an empty memory state, the robot begins with a selected prior collaboration pattern. In this way, the contribution of the method is not only better memory organization, but also a robot that can enter a new interaction with more useful prior team knowledge. Because the reused experience remains an explicit CP rather than opaque policy parameters, the approach also preserves an inspectable link between prior teamwork and later robot behavior.

To summarize, our main contributions are as follows:

- **Episodic Team Memory Representation:** We represent collaboration patterns (CPs) from prior human-robot interactions as knowledge-graph episodic memories, preserving situational context, action structure, and task outcomes in a form that can be reused across episodes.
- **Selecting Reusable Prior Experience:** We use graph representation learning with node-type supervision to organize historical CPs and identify a representative prior collaboration pattern to initialize the robot before a new interaction begins.
- **Improved Early-Team Performance:** Initializing the robot with a selected prior memory raises success by 60.7%, reduces average task time by 283 seconds, and yields the largest gains at the beginning of collaboration, when teams are still orienting to the task.

II. Background

A. MATRX Urban Search and Rescue (USAR)

The MATRX Urban Search and Rescue (USAR) environment [9], [10] simulates a USAR scenario where a collaborative virtual robot and a human must work together to rescue a victim. Figure 1 provides an example screenshot of the environment, while Figure 2 illustrates the features of the user interface used for human-robot communication. The CPs were originally stored in TypeDB [14], which uses an entity-relation model for human-robot interactions.

The team objective is to free a buried victim by removing rubble in front of the victim and opening an access route from one side, while minimizing elapsed time and avoiding additional harm from falling rocks [10]. The task is intentionally asymmetric: the robot can manipulate both small and large rocks, whereas the human is more limited physically but can better anticipate side effects such as rock falls and unsafe removals [10], [15]. This asymmetry makes the task genuinely collaborative rather than a simple division of identical actions.

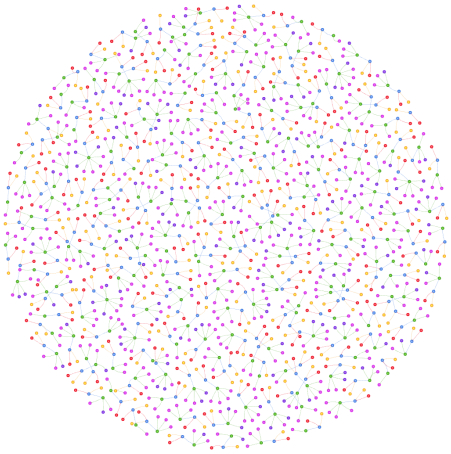
The participants begin with a practice round, followed by eight real rounds of varying difficulty. More specifically, the study design uses five runs of an easier level, followed by three runs of a harder level that introduces a brown rock whose placement can make rescue impossible if handled poorly [10]. This repeated structure was designed to let human and robot adapt over time, so that emergent strategies and collaboration patterns can be observed across rounds [10], [15]. Participants can define any CPs they find necessary, which remain available throughout the rounds unless they choose to modify or remove them. Later work added reflective communication support so that teams could explicitly externalize and discuss such emergent collaboration patterns through a shared ontology and chat interface [11], [12].

B. Knowledge Graphs (KGs)

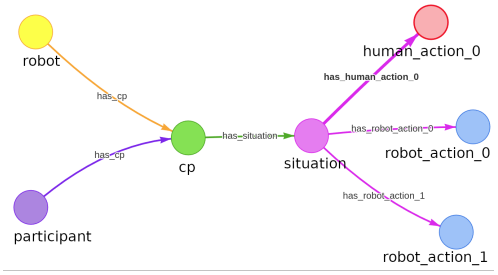
A KG is a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{R}, \mathcal{E})$, where $\mathcal{V}$ represents entities, $\mathcal{R}$ denotes relations, and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{R} \times \mathcal{V}$ consists of edges that define the relationships between entities. Each edge (v_i, r_k, v_j) indicates that a relation r_k exists from entity v_i to entity v_j [16]. While RDF graphs emphasize subject-predicate-object triples and interoperability [17], we use a property graph model implemented in JanusGraph [18], which allows both nodes and edges to carry attributes and supports efficient querying with Gremlin, making it well-suited for capturing the situational and action structure of CPs.

C. Graph Neural Networks (GNNs)

GNNs are deep learning models designed for graph-structured data [19]. They update each node's representation by aggregating features from its neighbors through successive message-passing layers, making them well-suited for tasks involving complex relational structures such as KG reasoning and graph representation learning. In our setting, this means that a node representation can reflect not only local attributes but also the surrounding situation and action context through neighborhood-based graph convolution. In the original Graph Convolutional Network (GCN), neighbor information is aggregated without distinguishing edge or relation types [20]. Relational Graph Convolutional Networks (RGCNs) extend this idea by introducing relation-specific transformations for typed edges [21]. For an RGCN layer, a



(a) A visualization of collected 209 collaboration patterns (CPs) as 209 knowledge graphs (KGs).



(b) A visualization of an example CP. This knowledge graph (KG) has seven nodes: one `robot`, `participant`, `cp`, and `situation`, along with one `human_action` and two `robot_action` nodes. While all CPs share the same core structure, the number of `human_action` and `robot_action` nodes varies, as annotators define action sequences for the human and robot. This graph representation naturally accommodates such variable-length collaborative routines.

Fig. 3: Visualizations of collaboration patterns (CPs) represented as knowledge graphs (KGs). (a) The entire set of 209 CPs. (b) An example CP illustrating its structural components.

standard update is

$$\mathbf{h}_v^{(\ell+1)} = \sigma\left(\sum_{r\in\mathcal{R}}\sum_{u\in\mathcal{N}_v^r}\frac{1}{c_{v,r}}\mathbf{W}_r^{(\ell)}\mathbf{h}_u^{(\ell)} + \mathbf{W}_0^{(\ell)}\mathbf{h}_v^{(\ell)}\right),$$

where $\mathbf{h}_v^{(\ell)}$ is the representation of node v at layer ℓ, $\mathcal{N}_v^r$ denotes the neighbors connected to v by relation type r, $\mathbf{W}_r^{(\ell)}$ is a relation-specific weight matrix, $c_{v,r}$ is a normalization constant, $\mathbf{W}_0^{(\ell)}$ is the weight matrix for the self-node contribution, and σ is a nonlinearity. In our setting, the edge labels serve as relation types that determine how messages are passed; we do not learn separate edge-state updates beyond these relation-specific transformations.

III. Methodology

A. Ontology Engineering

Because CP graphs have variable topology, we represent them as property graphs rather than force-fitting them into fixed-length Euclidean vectors, thus retaining full relational context.

Our ontology defines six entity types: `robot`, `participant`, `cp`, `situation`, `robot_action`, and `human_action`. Each entity is represented as a node with associated properties. Relationships between entities are captured through four edge families: `has_cp`, `has_situation`, `has_robot_action`, and `has_human_action`, each of which can also store relevant properties. For descriptive summaries, we annotate action edges with their sequence index (e.g., `has_robot_action_0`) to show their order within a CP. Conceptually, however, these are still instances of the same two action-edge families: human-action edges always link a CP to a human action, and robot-action edges always link a CP to a robot action. In other words, the index tells us whether an action happened first, second, or later, but it does not change the basic meaning of the edge in the ontology.

For graph learning, we make a more fine-grained distinction and treat these indexed action relations as different relation labels. The reason is simple: the encoder should be able to tell apart an early action from a later one, since action order is part of the collaboration pattern. If all robot-action edges shared one label, then "first robot action" and "later robot action" would look identical to the RGCN. Therefore, the encoder uses nine directed relation labels in total: one `has_situation` label, three `has_human_action_k` labels, and five `has_robot_action_k` labels.

From the previous studies, we have collected 209 CPs. Figure 3 visualizes them. We used the JanusGraph-Visualizer [18].

Table I lists the node properties of the example CP visualized in Figure 3b. The `cp` node properties "time_elapsed", "remaining_rocks", "victim_harm", and "success", serve as round-level performance metrics. While not CP-specific, we assume that if the human-robot team performs well in a round, the CPs used contribute to that success.

B. Graph Representation Learning on CP Graphs

Choosing a CP by raw score alone ignores the situational and action context that produced that score. A high-performing CP in one scenario may not generalize to others. To address this, we cluster CPs using graph representation learning, capturing both performance and structural similarities. This enables us to identify a representative and effective CP for reuse.

Since our CPs are structured as KGs, we use a Relational Graph Convolutional Network (RGCN) [21] to learn meaningful representations. Unlike standard GNNs, RGCN incorporates edge types (e.g., `has_cp`, `has_human_action`), which capture essential relational information. Through message passing, RGCN iteratively updates node features, allowing interactions between entities to shape their representations. We train the encoder with a node-classification objective by adding a single linear classifier on top of the final RGCN layer. Each node belongs to one of ten predefined

Type	Properties
robot	
participant	participant_number: 4,081
cp	time_elapsed: 3,000, event_time: "2024-05-07T11:26:11", cp_num: 36, ticks_lasted: 1,917, participant_num: 4,081, remaining_rocks: 32, cp_name: "top brown", victim_harm: 0, success: false round_num: 6
situation	location: "Top of rock pile", object: "Brown rock"
human_action_0	location: "Above rock pile", action: "Stand still in <location>"
robot_action_0	location: "Top of rock pile", object: "Brown rock", action: "Pick up <object>in <location>"
robot_action_1	location: "<Right>side of field", object: "Brown rock", action: "Drop <object>in <location>"

TABLE I: The node properties of the example CP.

classes used in our dataset: one cp class, one situation class, three human-action-stage classes, and five robot-action-stage classes. Thus, our representation learning uses supervision derived from the CP schema rather than a fully self-supervised objective. The message-passing mechanism ensures that gradients propagate beyond individual nodes, influencing their neighbors and refining the entire graph representation. In the implementation, the encoder itself is nonlinear because a ReLU activation is applied between the first and second RGCN layers; the input projection and the final classification head are linear.

As shown in Table I, node properties contain various features, requiring transformation into fixed-length vectors for RGCN compatibility. For cp nodes, we concatenate six numerical features ("time_elapsed", "ticks_lasted", "remaining_rocks", "victim_harm", "success", and "round_num"), normalizing each by its maximum value to scale them between 0 and 1. For situation, human_action, and robot_action nodes, we concatenate string attributes into natural language sentences (e.g., human_action_0 in the example would be "Action: Stand still in <location>. Location: Above rock pile.") and encode them using Sentence Bidirectional Encoder Representations from Transformers (Sentence-BERT) [22] to obtain numerical vector representations. Since the cp node vectors are shorter than the others, we apply a learnable projection matrix to transform them to the same dimensionality as the vectors for situation, human_action, and robot_action nodes. Concretely, if node v is a low-dimensional cp node with input $\mathbf{x}_v \in \mathbb{R}^{d_s}$, we first project it as $\tilde{\mathbf{x}}_v = \mathbf{W}_{\text{up}}\mathbf{x}_v$, where $\mathbf{W}_{\text{up}} \in \mathbb{R}^{d_b \times d_s}$ maps it into the common d_b-dimensional feature space used by the other node types. This is a single linear projection, not a deeper MLP.

We optimize RGCN using cross-entropy loss for node classification. Let f_θ represent the entire model, including RGCN and the linear classifier. Given a CP (KG) $\mathcal{G}$ with node-feature matrix X and graph connectivity A, the predicted class distribution for a node v is computed from the final RGCN embedding $\mathbf{h}_v^{(2)}$ as $\mathbf{z}_v = \mathbf{W}_{\text{cls}}\mathbf{h}_v^{(2)}$ and $\hat{\mathbf{y}}_v = \text{softmax}(\mathbf{z}_v)$, where $\mathbf{W}_{\text{cls}}$ denotes the single linear classification head. Equivalently, f_θ denotes the full model consisting of the input projection, two RGCN layers, and this classifier. The loss function is:

$$\mathcal{L} = -\mathbb{E}_{(v,c)\sim\mathcal{D}_{\text{train}}}\left[\log \hat{y}_{v,c}\right]$$

where $\mathcal{D}_{\text{train}}$ represents the empirical distribution of training samples, $\hat{y}_{v,c}$ denotes the predicted probability assigned to the true class c of node v, and C is the number of node classes.

The message-passing mechanism propagates updates across the graph, refining node representations based on their connectivity. Action stages are treated as distinct node classes so that the encoder can distinguish early from later steps within a collaboration pattern. In total, we use $C = 10$ node classes. See Figure 4 for a visualization of one forward pass of the RGCN-based neural network. After training, we obtain a graph-level embedding for each CP graph $\mathcal{G}_i = (\mathcal{V}_i, \mathcal{E}_i)$ by mean pooling its final node embeddings,

$$\mathbf{g}_i = \frac{1}{|\mathcal{V}_i|}\sum_{v\in\mathcal{V}_i}\mathbf{h}_v^{(2)},$$

where $|\mathcal{V}_i|$ is the number of nodes in graph $\mathcal{G}_i$. These pooled graph embeddings are the vectors that we cluster with K-means.

After this representation-learning phase, we use the learned graph representations to cluster the CPs. From these clusters, we select one centroid-nearest representative CP, which we then use to initialize the robot's episodic memory. This selected CP is not a learned control policy. Rather, it is a structured memory item containing a situation and an associated action sequence that is preloaded before a new trial begins. During execution, the robot continues to use the existing MATRX control framework, but it can reuse the preloaded CP when the current situation matches the stored condition. We then conduct human-robot interaction experiments in the same MATRX collaborative environment to evaluate the impact of this memory augmentation on task performance metrics. An additional advantage of this design is transparency: the transferred prior remains a human-readable situation-action structure that can be inspected, discussed, and revised, rather than a latent policy update that is difficult for teammates or designers to interpret.

IV. Experiments

The two-layer RGCN (128,874 parameters, ReLU activations) trained for 2,000 epochs on all 209 CP graphs in a single 30s CPU run; code and hyper-parameters are in the

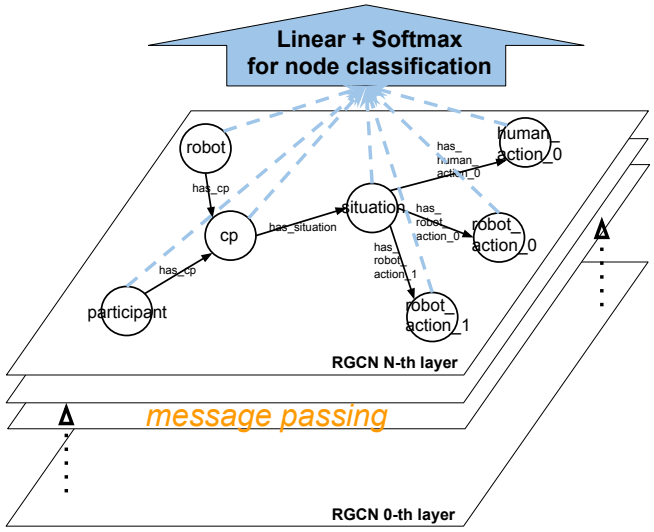


Fig. 4: A forward pass of the RGCN-based neural network. Low-dimensional `cp` node features are first linearly projected to the common feature space, and a single linear classifier with softmax is applied to the final RGCN node embeddings to compute the cross-entropy loss.

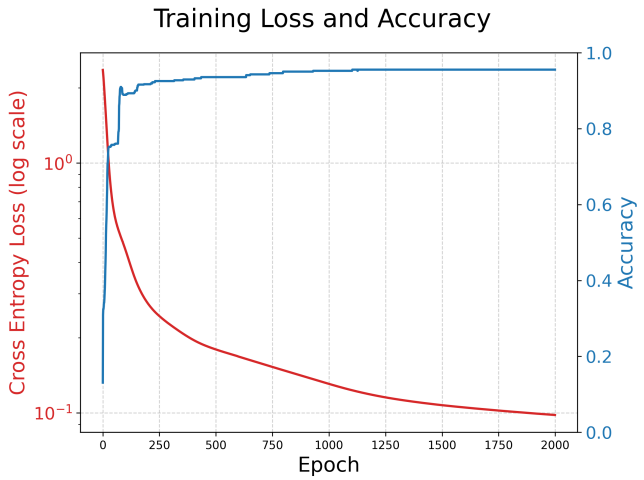
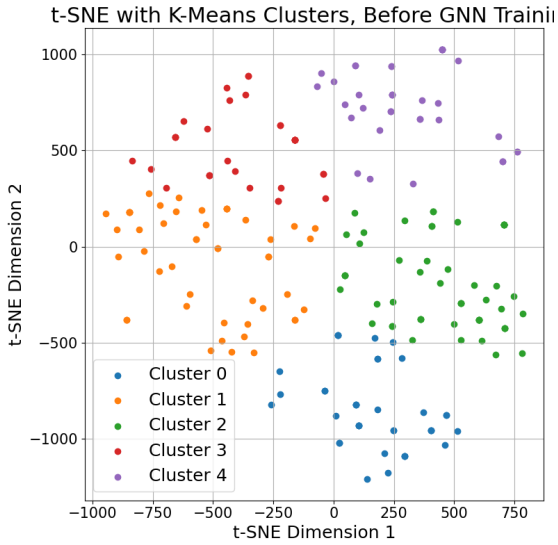


Fig. 5: Cross entropy loss and accuracy of the training data. After 2,000 epochs, the loss dropped to 0.0948 while the accuracy went up to 95.5%.



(a) 5 clusters of CPs before training. Since there are not yet GNN-based vector representations for the CPs, we used the `cp` node features as vectors.



(b) 5 clusters of CPs after training. The vectors used here are the means of the last RGCN layer node representations.

Fig. 6: K-means clustering results before and after training, visualized using t-SNE. $K = 5$ was chosen.

supplementary material, along with a recorded game play and raw data. See Figure 5 for the training loss, which steadily decreases over epochs, while the accuracy increases, indicating effective learning and convergence of the model.

After training, we obtain the final vector representation of each CP by averaging the node representations (before the classification layer) within each graph. We then apply the K-means clustering algorithm to group CPs based on Euclidean distance. Figure 6 illustrates the clusters before and after training, with t-distributed stochastic neighbor embedding (t-SNE) [23] used for 2D visualization. The learned RGCN representations produce clusters that are more interpretable than those obtained from raw `cp` node features alone. The number of CPs in the clusters are 62, 38, 36, 30, and 43, respectively. We set $K = 5$, where K is the number of K-means clusters, based on exploratory inspection of cluster interpretability and balance; this was a heuristic design choice rather than a tuned optimum.

Table II presents the performance metrics for each cluster. Notably, Cluster 1 exhibits a higher success rate, suggesting that participants using CPs from this cluster were more likely to rescue the victim. While the other metrics remain important, we emphasize success rate since saving the victim is considered the most critical outcome.

Table III presents the edge types and their frequencies for

Cluster	Time Elapsed	Remaining Rocks	Victim Harm	Success Rate
Cluster 0	2,716 (+120)	16 (+1.0)	291 (-14)	24% (-3%)
Cluster 1	2,133 (-462)	11 (-3.9)	318 (+11)	42% (+15%)
Cluster 2	2,607 (+12)	11 (-3.2)	311 (+4)	36% (+9%)
Cluster 3	2,869 (+273)	18 (+3.9)	246 (-60)	17% (-11%)
Cluster 4	2,629 (+33)	16 (+1.9)	355 (+49)	19% (-9%)

TABLE II: Cluster statistics (higher is better for success rate and lower is better for the others.), with differences from the mean in parentheses.

Cluster	Edge label	Count (%)
Cluster 0	has_robot_action_0	62 (55.86%)
	has_human_action_0	30 (27.03%)
	has_human_action_1	8 (7.21%)
	has_robot_action_2	3 (2.70%)
	has_robot_action_3	3 (2.70%)
	has_robot_action_4	3 (2.70%)
	has_human_action_2	2 (1.80%)
Cluster 1	has_robot_action_1	38 (48.72%)
	has_robot_action_0	38 (48.72%)
	has_robot_action_2	2 (2.56%)
Cluster 2	has_robot_action_1	36 (31.30%)
	has_robot_action_0	36 (31.30%)
	has_robot_action_2	31 (26.96%)
	has_robot_action_3	7 (6.09%)
	has_human_action_1	4 (3.48%)
	has_human_action_2	1 (0.87%)
Cluster 3	has_human_action_0	30 (38.46%)
	has_robot_action_1	30 (38.46%)
	has_robot_action_2	16 (20.51%)
	has_robot_action_3	2 (2.56%)
Cluster 4	has_human_action_0	43 (26.22%)
	has_robot_action_1	43 (26.22%)
	has_robot_action_0	42 (25.61%)
	has_human_action_1	20 (12.20%)
	has_robot_action_2	7 (4.27%)
	has_robot_action_3	7 (4.27%)
	has_human_action_2	2 (1.22%)

TABLE III: Edge label distributions across clusters.

the CPs in each cluster. Notably, the action edges in Cluster 1 are simpler than those in other clusters, as it contains fewer distinct action types. Of the 78 total edges across all CPs in Cluster 1, the majority involve two consecutive robot actions. This suggests that Cluster 1 not only represents a simpler interaction pattern but also grants greater autonomy to the robot. While robot actions are typically conditioned on human actions, the lower presence of human action edges in this cluster indicates that the robot's behavior is less dependent on human input.

Considering both success rate and structural simplicity, we heuristically selected Cluster 1 as the cluster from which to draw a reusable CP. It represents a minimal yet effective interaction pattern, where the robot operates with greater autonomy while maintaining efficiency. We then selected the CP closest to the cluster center (by Euclidean distance) as our final choice, using it as the centroid-nearest representative of that cluster as a proof-of-concept exemplar-selection procedure rather than claiming it to be globally optimal.

Type	Details
`situation`	Location: "Top of rock pile" Object: "Large rock"
`robot_action_0`	Location: "Top of rock pile" Object: "Large rock" Action: "Pick up <object> in <location>"
`robot_action_1`	Location: "<Right> side of field" Object: "Large rock" Action: "Drop <object> in <location>"

TABLE IV: The final selected collaboration pattern (CP).

Table IV presents the final selected CP. Notably, this CP consists of only two consecutive robot actions without any human actions. This suggests that, in this scenario, the robot operates independently, executing its tasks without relying on human input. Compared to other CPs that typically involve human actions, this CP grants the robot greater autonomy in decision-making and task execution. Specifically, this CP directs the robot to move large rocks near the victim. Since only the robot can handle large rocks, this pattern effectively delegates the heavy lifting to the robot while allowing the human to focus on other tasks as they see fit.

We initialize the robot's memory with the final CP and conduct the MATRX USAR experiment under the same conditions as the previous study, with the only difference being the inclusion of the preloaded memory. The historical memory pool contains 209 previously collected CPs from earlier MATRX studies and is treated here as a fixed offline memory source. The robot is initialized with only one selected CP at test time. The same selected CP is reused for all participants in the memory-initialized condition, so the evaluation tests transfer of one shared prior team experience rather than per-user personalization. We recruited 20 student participants from our university. All experiments were conducted online.[1]

Table V shows that the robot with initialized memory consistently outperforms the one without it, except in "victim_harm," likely due to accidental rock falls when removing large rocks. Across the 160 round-level observations, the success rate increased from 25.7% to 41.3% (a 60.7% relative improvement). We treat this comparison as descriptive evidence because rounds are repeated within participants. Notably, the most significant improvement occurs in Round 1, where participants are still unfamiliar with the environment. This early-round pattern is important because it suggests that the benefit comes before substantial within-session human adaptation can occur. Since the robot begins with prior knowledge from past interactions, it can immediately

[1]The experiment involving human participants was approved by our university's ethics committee (ID anonymized).

Round	Time Elapsed		Remaining Rocks		Victim Harm		Success Rate	
	W/o Memory Initialized	With Memory Initialized	W/o Memory Initialized	With Memory Initialized	W/o Memory Initialized	With Memory Initialized	W/o Memory Initialized	With Memory Initialized
1	3,000 (0)	2,458 (626)	18.2 (6.0)	5.8 (8.0)	733 (512)	160 (320)	0.0% (0.0%)	55.0% (51.0%)
2	2,677 (604)	2,304 (644)	14.5 (13.3)	5.3 (8.1)	324 (387)	255 (349)	28.5% (46.3%)	65.0% (48.9%)
3	2,410 (707)	2,192 (732)	10.4 (13.7)	3.5 (7.3)	257 (457)	465 (589)	52.4% (51.1%)	75.0% (44.4%)
4	2,475 (699)	2,150 (672)	7.9 (9.5)	3.5 (7.0)	286 (408)	160 (201)	47.6% (51.2%)	75.0% (44.4%)
5	2,493 (722)	2,300 (720)	15.9 (12.0)	12.0 (9.5)	457 (383)	470 (508)	23.8% (43.6%)	30.0% (47.0%)
6	2,590 (630)	2,593 (545)	19.0 (9.6)	14.0 (7.7)	476 (648)	520 (393)	4.7% (21.8%)	15.0% (36.6%)
7	2,503 (555)	1,994 (784)	15.5 (11.9)	17.1 (6.9)	310 (319)	880 (635)	23.8% (43.6%)	5.0% (22.4%)
8	2,558 (545)	2,448 (660)	14.5 (11.4)	12.5 (6.9)	535 (701)	1,190 (679)	25.0% (44.4%)	10.0% (30.8%)
Average	2,588 (613)	**2,305 (686)**	14.5 (11.4)	**9.2 (9.1)**	**422 (504)**	513 (583)	25.7% (43.9%)	**41.3% (49.4%)**

TABLE V: Comparison of the robot without initialized memory and the robot with initialized memory. Values are presented as *Average (std)*. Success rate is reported as a percentage (%), where higher is better; lower is better for the other metrics. Better values for each metric are shown in **bold**.

engage in effective collaboration, providing a smoother start for the human-robot team. This matters because the initial rounds are precisely when shared routines have not yet been established; helping the team start in a more coordinated state can therefore reduce the early coordination cost.

Interestingly, the selected CP (Table IV) does not reference the "Brown rock," which only appears in later rounds (Rounds 5–8). Despite this, the robot with initialized memory continues to facilitate better performance in most of the rounds, except 7 and 8. This suggests that the chosen CP captures a generally effective collaboration strategy, enabling the robot to adapt even in scenarios it has not explicitly encountered before.

V. Related Work

Our work sits at the intersection of memory-based agents, human-robot teaming, co-learning, and prior MATRX USAR research. Below, we position the paper relative to these strands and emphasize where our contribution differs.

Prior work on episodic memory and continual learning has shown that storing and revisiting past experience can support faster adaptation and incremental improvement [24], [25]. Related agent-memory architectures have also separated episodic from semantic memory and, more recently, represented temporally evolving knowledge in graph form for sequential decision-making [26], [27], [28]. These works motivate the broader idea that memory should help future decision-making, but they do not address the specific problem studied here: how to carry forward a prior human-robot collaboration pattern into a new team episode. Our work differs in both what is stored and how it is reused. Rather than retaining generic experiences or latent policy updates, we represent prior human-robot collaboration patterns as explicit knowledge-graph memories containing situation, action-order, and outcome information, and we select one representative memory for reuse at test time. The contribution is therefore not a general memory architecture, but a concrete mechanism for transferring an inspectable prior team experience into a new collaboration episode.

Research on human-robot teaming has emphasized mutual adaptation, shared understanding, team dynamics, trust, and broader human-autonomy teaming requirements [3], [5], [8], [4]. Related co-learning work has explored user feedback, models of human decision making, reinforcement learning, and transparency in interactive AI systems [29], [30], [31]. These studies explain why robots should adapt to human partners, but they typically focus on adaptation during interaction. In contrast, our emphasis is on pre-interaction transfer: the robot begins with a selected prior collaboration pattern before meeting a new partner. This distinction matters because our method targets the earliest stage of collaboration, when shared routines have not yet formed. In that sense, the paper is less about learning a better online policy during a session and more about improving how the robot enters the session in the first place.

Within urban search and rescue, prior MATRX-based studies showed that teams can externalize collaboration patterns through ontology-supported interfaces and chat, and that these patterns are useful for analyzing human-robot co-adaptation over repeated rounds [11], [10], [12]. Complementary work has also examined how rescuers perceive robotic teammates in disaster scenarios [7]. Our work builds directly on this line, but changes the role of CPs: instead of treating them only as reflective artifacts or descriptive interaction traces, we turn them into reusable episodic memories. We then use graph representation learning to cluster these memories and select a representative CP for reuse, linking prior collaboration analysis to improved performance in later human-robot teamwork. This is the key distinction from the earlier MATRX studies: the same ontology-backed CPs are no longer only analyzed after interaction, but are reused before interaction to shape future collaboration.

VI. Conclusion and Limitations

We showed that initializing a USAR robot with a single prior collaboration pattern stored as a knowledge-graph episodic memory can improve human-robot teamwork. Across 20 participants and 160 round-level observations, the memory-initialized robot raised victim-rescue success from 25.7% to 41.3% and cut average task time by 283 seconds, with the largest gains appearing early in interaction.

These results suggest that robots can benefit from entering a new task with reusable knowledge distilled from earlier collaborations rather than starting from an empty memory state. More specifically, the contribution is a simple test-time memory initialization mechanism: prior team experience is encoded once, selected offline, and reused at the start of a new collaboration episode. More broadly, the paper argues for a view of robot learning that is compatible with human-robot symbiosis: prior experience should not only improve behavior, but remain inspectable enough to support communication, shared understanding, and iterative refinement by human teammates.

The study nevertheless has limitations. It was conducted in simulation with 20 student participants, used one centroid-nearest representative CP drawn from a heuristically selected cluster, and relied on a domain-specific ontology; budget constraints also precluded testing other cluster exemplars, and victim harm sometimes increased when the robot moved heavy debris. Future work will examine online memory growth and pruning, compare multiple candidate memories, test alternative cluster choices, and evaluate whether reusable episodic team memory also improves human-centered outcomes such as trust, coordination quality, cognitive load, and perceived fluency. In physically and mentally demanding contexts like disaster response, this kind of pre-aligned structural task knowledge may reduce cognitive burden on human operators and therefore merits direct measurement in future studies.

REFERENCES

[1] IFR, "Record of 4 million robots working in factories worldwide," 2023. Accessed: 2025-02-03.

[2] MarketsandMarkets, "Collaborative robot market by payload, component, industry, and region - global forecast to 2030," 2024. Accessed: 2025-02-03.

[3] S. Nikolaidis, D. Hsu, and S. Srinivasa, "Human-robot mutual adaptation in collaborative tasks: Models and experiments," *The International Journal of Robotics Research*, vol. 36, p. 027836491769059, 02 2017.

[4] M. O. Smith, S. Amatya, A. Amresh, J. C. Gorman, M. Johnson, N. J. Cooke, and W. Zhang, "Research needs in human-autonomy teaming: Thematic analysis of priority features for testbed development," in *33rd IEEE International Conference on Robot and Human Interactive Communication, RO-MAN 2024, Pasadena, CA, USA, August 26–30, 2024*, pp. 1183–1190, IEEE, 2024.

[5] A. Haripriyan, R. Jamshad, P. Ramaraj, and L. D. Riek, "Human-robot action teams: A behavioral analysis of team dynamics," in *33rd IEEE International Conference on Robot and Human Interactive Communication, RO-MAN 2024, Pasadena, CA, USA, August 26–30, 2024*, pp. 1443–1448, IEEE, 2024.

[6] S. K. R. Moosavi, M. H. Zafar, and F. Sanfilippo, "Collaborative robots (cobots) for disaster risk resilience: a framework for swarm of snake robots in delivering first aid in emergency situations," *Frontiers in Robotics and AI*, vol. 11, p. 1362294, 2024.

[7] Z. Betta, A. Gaudino, A. Benini, C. T. Recchiuto, and A. Sgorbissa, "Perceptions and opinions of rescuers about a quadruped robot in an earthquake scenario," in *33rd IEEE International Conference on Robot and Human Interactive Communication, RO-MAN 2024, Pasadena, CA, USA, August 26–30, 2024*, pp. 1092–1099, IEEE, 2024.

[8] R. Chacón-Quesada, F. E. Casado, and Y. Demiris, "On the effect of augmented-reality multi-user interfaces and shared mental models on human-robot trust," in *33rd IEEE International Conference on Robot and Human Interactive Communication, RO-MAN 2024, Pasadena, CA, USA, August 26–30, 2024*, pp. 1316–1322, IEEE, 2024.

[9] T. H. Jasper van der Waa, "Matrx: Human agent teaming rapid experimentation software," July 2023.

[10] E. van Zoelen, K. Bosch, and M. Neerincx, "Becoming team members: Identifying interaction patterns of mutual adaptation for human-robot co-learning," *Frontiers in Robotics and AI*, vol. 8, 07 2021.

[11] E. M. van Zoelen, K. van den Bosch, and M. Neerincx, "Human-robot co-learning for fluent collaborations," in *Companion of the 2021 ACM/IEEE International Conference on Human-Robot Interaction*, HRI '21 Companion, (New York, NY, USA), p. 574–576, Association for Computing Machinery, 2021.

[12] E. van Zoelen, K. Bosch, D. Abbink, and M. Neerincx, *Ontology-Based Reflective Communication for Shared Human-AI Recognition of Emergent Collaboration Patterns*, pp. 621–629. 11 2022.

[13] L. Miraglia, C. D. Dio, F. Manzi, T. Kanda, A. Cangelosi, S. Itakura, H. Ishiguro, D. Massaro, P. Fonagy, and A. Marchetti, "Shared knowledge in human-robot interaction (hri)," *International Journal of Social Robotics*, vol. 16, p. 59–75, 2024.

[14] C. Dorn and H. Pribadi, "Typeql: A type-theoretic & polymorphic query language," *Proceedings of the ACM on Management of Data*, vol. 2, pp. 1–27, 05 2024.

[15] E. M. van Zoelen, *Human-Machine Co-Learning: Anticipating, Identifying and Sharing Emergent Collaboration Patterns*. PhD thesis, Delft University of Technology, 2025.

[16] A. Hogan, E. Blomqvist, M. Cochez, and et al., "Knowledge graphs," *ACM Comput. Surv.*, vol. 54, jul 2021.

[17] M. Lanthaler, D. Wood, and R. Cyganiak, "RDF 1.1 concepts and abstract syntax," W3C recommendation, W3C, February 2014. https://www.w3.org/TR/2014/REC-rdf11-concepts-20140225/.

[18] JanusGraph Contributors, "Janusgraph: an open-source, distributed graph database," 2024.

[19] F. Scarselli, M. Gori, A. C. Tsoi, and et al., "The graph neural network model," *IEEE Transactions on Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009.

[20] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *CoRR*, vol. abs/1609.02907, 2016.

[21] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," 2017.

[22] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," 2019.

[23] L. van der Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.

[24] A. Pritzel, B. Uria, S. Srinivasan, A. P. Badia, O. Vinyals, D. Hassabis, D. Wierstra, and C. Blundell, "Neural episodic control," in *Proceedings of the 34th International Conference on Machine Learning* (D. Precup and Y. W. Teh, eds.), vol. 70 of *Proceedings of Machine Learning Research*, pp. 2827–2836, PMLR, 06–11 Aug 2017.

[25] A. Chaudhry, M. Ranzato, M. Rohrbach, and M. Elhoseiny, "Efficient lifelong learning with a-gem," 2019.

[26] T. Kim, M. Cochez, V. Francois-Lavet, and et al., "A machine with human-like memory systems," 2022.

[27] T. Kim, M. Cochez, V. Francois-Lavet, and et al., "A machine with short-term, episodic, and semantic memory systems," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, Jun. 2023.

[28] T. Kim, V. François-Lavet, and M. Cochez, "Temporal knowledge-graph memory in a partially observable environment," 2026.

[29] R. Kumar, V. Som, and N. Yao, "Sequential transfer learning-based human decision making model for human-robot co-learning and insights from user feedback analysis," in *33rd IEEE International Conference on Robot and Human Interactive Communication, RO-MAN 2024, Pasadena, CA, USA, August 26–30, 2024*, pp. 250–257, IEEE, 2024.

[30] A. Shafti, J. Tjomsland, W. Dudley, and A. A. Faisal, "Real-world human-robot collaborative reinforcement learning," *CoRR*, vol. abs/2003.01156, 2020.

[31] J. Wenskovitch and C. North, "Interactive artificial intelligence: Designing for the "two black boxes" problem," *Computer*, vol. 53, no. 8, pp. 29–39, 2020.